

SIMATS ENGINEERING

ASSESSMENT TOOL 3

**COURSE NAME: Artificial Intelligence
CSA17**

COURSE CODE:

Name: Hemalakshmi H
Reg.No.: 192571049

Reverse Engineering of Intelligent Systems: Architecture Analysis, Algorithm Identification, Data Flow Mapping, and Redesign Strategies

1. System Decomposition & Analysis

Reverse engineering begins with decomposing an existing AI-based system into its core components to understand how it functions internally. The goal is to uncover the architecture, modules, data flow, and algorithms used.

Component Identification

A typical AI system contains:

- **Input Layer:** Collects raw data (user logs, sensor readings, images).
- **Preprocessing Layer:** Cleans, transforms, and normalizes data.
- **Model Layer:** Implements ML algorithms, feature extraction, and decision logic.
- **Output Layer:** Produces predictions, recommendations, alerts, or classifications.
- **Storage Layer:** Databases, distributed file systems, logs.
- **Control Layer:** Orchestration, scheduling, monitoring.

Data Flow Analysis

Data moves from input → preprocessing → model → output. Logs flow into storage → batch processing → model retraining → updated inference pipeline.

Algorithm Identification

By observing system behavior, algorithms such as clustering, regression, neural networks, or collaborative filtering can be inferred.

Outcome

A complete breakdown of the system's internal structure, enabling deeper analysis and reconstruction.

2. Architecture Reconstruction

Architecture reconstruction is performed when the system is a **black box** — internal code is hidden. The system is analyzed based on inputs, outputs, and behavior patterns.

Reconstructed Modules

Example: Fraud Detection System

- **Input Module:** Receives transaction streams.
- **Preprocessing Module:** Cleans data, extracts features.
- **Detection Engine:** Applies rule-based filters and ML anomaly detection.
- **Decision Module:** Computes fraud probability and triggers alerts.
- **Output Module:** Sends notifications and generates reports.
- **Feedback Module:** Uses confirmed fraud cases to retrain models.

Module Interaction

Input → Preprocessing → Detection → Decision → Output Feedback loop → Model retraining → Improved detection accuracy.

Diagram Explanation

Block diagrams and flowcharts illustrate:

- Data movement
- Control flow
- Module dependencies
- Integration points

Outcome

A reconstructed architecture that explains how the system operates internally.

3. Algorithm Identification

Reverse engineering a machine learning application requires identifying the algorithms and decision logic used.

Preprocessing Algorithms

- Normalization
- Noise removal
- Data augmentation
- Tokenization (for text)
- Image resizing (for vision systems)

Feature Extraction Algorithms

- CNN filters (for images)
- Pooling layers
- TF-IDF / word embeddings (for text)
- Statistical feature extraction (for tabular data)

Core ML Algorithms

Depending on system behavior:

- **Classification:** Softmax, SVM, decision trees
- **Clustering:** K-means, DBSCAN
- **Regression:** Linear/Logistic regression
- **Deep Learning:** CNNs, RNNs, transformers
- **Anomaly Detection:** Isolation Forest, autoencoders

Decision Logic

- Thresholding
- Confidence scoring
- Rule-based filtering

- Ensemble voting

Outcome

A clear identification of the algorithms and logic powering the system.

4. Data Flow & Process Mapping

Mapping data flow reveals how information moves through a distributed AI system and how components communicate.

Data Sources

- IoT sensors
- Satellite feeds
- Social media streams
- Logs and databases
- External APIs

Ingestion Layer

- Kafka streaming
- REST API endpoints
- Batch uploads
- Message queues

Processing Pipeline

- PySpark RDD/DataFrame transformations
- Feature extraction
- Spatial-temporal fusion
- ML inference
- Aggregation and scoring

Communication Between Components

- Sensor agents → Ingestion layer
- Ingestion → Processing engine

- Processing → Decision agents
- Decision → Alert/communication agents
- Communication → Public dashboards, authorities

Output Layer

- Risk maps
- Alerts
- Reports
- Predictions

Outcome

A complete map of how data flows from source to decision-making modules.

5. Improvement & Redesign

Reverse engineering reveals weaknesses and opportunities for improvement. Redesign strategies enhance scalability, efficiency, and performance.

Identified Issues

- High latency
- Slow batch processing
- Poor scalability
- Inefficient algorithms
- Limited real-time capability
- Bottlenecks in communication

Proposed Enhancements

1. Scalability Improvements

- Shift from monolithic to microservices
- Use distributed storage (HDFS, Cassandra)
- Introduce load balancing
- Parallelize ML training

2. Efficiency Improvements

- Optimize RDD/DataFrame operations
- Use caching and in-memory computation
- Replace slow algorithms with faster variants
- Reduce redundant data transformations

3. Performance Improvements

- GPU acceleration for ML models
- Asynchronous agent communication
- Real-time streaming instead of batch processing
- Improved indexing and query optimization

4. Redesign Strategies

- Introduce multi-agent coordination
- Add reinforcement learning for adaptive optimization
- Containerize system using Docker/Kubernetes
- Improve fault tolerance and recovery mechanisms

Outcome

A redesigned intelligent system with improved speed, scalability, accuracy, and reliability.

Conclusion

This Reverse Engineering Assessment demonstrates the ability to analyze, decompose, reconstruct, and improve intelligent systems across multiple domains. By examining architecture, algorithms, data flow, and redesign strategies, the report showcases a deep understanding of how AI systems operate internally and how they can be enhanced for real-world performance.